

AI Debugger Pro

Reproducible Engineering and Release Evidence

T. R. Bentley | Paper 12 of 12 | September 2026

ABSTRACT

Software claims become credible when readers can trace them to source, environment, raw observations, explicit derivations, and archived artifacts. This paper defines a practical evidence graph and release gate for AI Debugger Pro. It distinguishes measured results, implemented controls, and planned validation for independent review.

Paper 12 of 12 | 1

1. Motivation and Baseline

Reproducibility makes a result auditable. The current measured baseline is Paper 8's frozen 24-case benchmark and its recorded 27-test CI baseline. These observations are not universal guarantees. This paper specifies controls for reproducing those measurements and strengthening later releases without inventing additional outcomes.

2. Evidence Graph

source commit -> environment -> frozen inputs -> raw observations

-> derivation procedure -> published claim -> release artifact -> replication

Each edge needs a stable identifier, checksum, version, or documented transformation.

A broken edge lowers confidence even when the final chart appears plausible.

3. Reproducibility Levels

R0 Narrative only; R1 versioned source; R2 captured environment;

R3 frozen inputs, commands, logs, and outputs; R4 deterministic publication;

R5 independent replication with recorded variance.

The repository spans multiple levels. Included manifests expose current status without presenting proposed controls as completed evidence.

4. Artifact and Claim Manifests

The artifact manifest records IDs, roles, paths, formats, producers, checksum expectations, retention policies, and status. The claim-evidence matrix links statements to supporting artifacts and labels each measured, implemented, or hypothesis/planned.

Machine-readable CSV files allow automated release and audit checks.

5. Environment and AI Provenance

Capture runtime, dependency versions, platform, hardware class, locale, time settings, configuration, and exact commands. AI runs additionally need provider/model identifiers, prompt-template versions, sampling settings, tool permissions, context policy, retries, and redaction rules. Nondeterministic services require distributions, not claims of identity.

6. Reproducing Benchmarks and Figures

Freeze case definitions, expected behavior, evaluation rules, seeds, logs, and result tables.

Generate figures and manuscript tables from recorded data with deterministic procedures.

The gate verifies that claims resolve to data, figures resolve to derivations, and artifacts belong to the declared commit.

7. Release Evidence Gate

A release identifies source, dependencies, tests, inputs, raw outputs, derivations, documents, checksums, licensing, and limitations. Failures block release or become explicit exceptions. Platform-specific claims need platform-specific evidence. Signed tags, immutable archives, and retention rules preserve the chain after publication.

8. Replication and Series Integration

Replication records operator, platform, commit, procedure, deviations, outputs, and comparison.

Papers 1-7 cover system and methods; 8-9 add evaluation and failure taxonomy; 10-11 add context relevance and independent test oracles; Paper 12 binds the series to reproducible release evidence.

9. Roadmap and Conclusion

Near term: CI manifest validation, checksums, and data-driven publication.

Next: hermetic environments, multi-platform execution, and immutable release archives.

Later: blinded independent replication and longitudinal drift reporting.

The twelve-paper architecture is complete. Its claims remain bounded by recorded evidence, and this finale supplies the contract for durable, inspectable engineering knowledge.

Paper 12 of 12 | 10